

FRONTEND COHORT · VOLUME 01

HTML

the beginner's reference

• • •

A COMPANION GUIDE FOR

Tasty Bites — Web Development from Scratch

Everything you need for Lessons 01 & 02, plus tips you'll keep using forever.

CONTENTS

What's inside

A complete tour of HTML for absolute beginners — the grammar, the tags you'll use every day, and the little tricks that make pages feel polished.

01	What HTML actually is	<i>page 3</i>
02	Tags, elements, and attributes	<i>page 4</i>
03	The page skeleton	<i>page 6</i>
04	The everyday tags	<i>page 7</i>
05	Lists — bullets and numbers	<i>page 11</i>
06	Semantic HTML	<i>page 12</i>
07	div, span, and friends	<i>page 14</i>
08	Forms — letting users type	<i>page 15</i>
09	Tables — for actual tabular data	<i>page 19</i>
10	Multi-page sites	<i>page 20</i>
11	Tips & little hacks	<i>page 21</i>
12	Common mistakes	<i>page 25</i>
13	The complete tag cheat sheet	<i>page 26</i>
14	Where to go next	<i>page 28</i>

CHAPTER 01

What HTML actually is

If a webpage were a person, HTML would be the skeleton — the structure under everything you see.

HTML stands for **HyperText Markup Language**. It's not really a "programming" language — you don't write logic or do math in it. You describe what's on a page: *here is a heading, here is a paragraph, here is an image*. The browser reads your description and draws the result.

Every website on the internet — Google, YouTube, your bank's homepage — starts as HTML. The colors, fonts, and layout you see come from CSS. The interactivity (buttons that respond, forms that validate) comes from JavaScript. But the structure underneath? Always HTML.

HTML is to a webpage what the script is to a movie. Not the costumes, not the lighting — the words that tell the story.

How a browser reads HTML

When you save a file ending in `.html` and open it in a browser, the browser reads it top to bottom, builds a model of the page in memory (the DOM — Document Object Model), and paints it on screen. Change the file, refresh the browser, see the new version. That's the entire feedback loop.

QUICK MENTAL MODEL

HTML is the noun. CSS is the adjective. JavaScript is the verb. *"Show a red button that calls my friend."* Red = CSS, button = HTML, calls = JavaScript.

You need almost nothing to start

- A code editor — [VS Code](#) is free and what professionals use
- A web browser — Chrome, Edge, Firefox, Safari all work
- A folder anywhere on your computer to save your files in

No installation of HTML itself. The browser already speaks it.

— end of chapter one —

CHAPTER 02

Tags, elements, and attributes

Three words that get used almost interchangeably — but each one means something slightly different.

A tag is a word in angle brackets

```
<p>Hello world</p>
```

That whole thing — opening tag, content, closing tag — is an **element**. The element's name is `p` (for paragraph). Most elements come in pairs: an opening tag `<p>` and a closing tag `</p>` with content in the middle.

A few tags close themselves

Some elements have no content — they exist as one tag. The classic example:

```

```

You'll also see them written as `<img ... />` with a slash before the closing bracket. Both are valid. Other self-closing tags: `<br>`, `<hr>`, `<input>`, `<link>`, `<meta>`.

Attributes — extra info on a tag

In the image above, `src` and `alt` are **attributes**. They go inside the opening tag and follow this pattern:

```
name="value"
```

You'll meet dozens of attributes, but they all follow that shape. The most important ones early on:

ATTRIBUTE	USED ON	WHAT IT DOES
<code>href</code>	<code><a></code>	The URL the link points to
<code>src</code>	<code></code> , <code><script></code>	Source — where the file lives
<code>alt</code>	<code></code>	Text shown if the image fails (and read aloud by screen readers)
<code>id</code>	Any tag	A unique label for that one element
<code>class</code>	Any tag	A reusable label, used by CSS and JS
<code>title</code>	Any tag	Tooltip text on hover

Nesting — tags inside tags

Tags wrap around each other. The only rule is: close them in reverse order of opening.

```
<!-- Right -->  
<p>I love <strong>chocolate cake</strong> a lot.</p>  
  
<!-- Wrong -->  
<p>I love <strong>chocolate</p> cake</strong>
```

Browsers are forgiving — they'll usually render the wrong version anyway — but messy nesting causes bugs that take hours to find. Always close tags in the right order.

A NOTE ON QUOTES

Attribute values can use `"double"` or `'single'` quotes — pick one and stick with it. Double quotes are conventional.

Comments — notes the browser ignores

```
<!-- This is a comment. The browser ignores it. -->  
<!-- Use them to leave yourself notes or hide code temporarily. -->
```

— end of chapter two —

CHAPTER 03

The page skeleton

Every HTML file you ever write starts with the same handful of lines. Type them once, memorize them, move on.

The starter template

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My Page</title>
  </head>
  <body>
    <!-- Everything you see on the page goes here -->
  </body>
</html>
```

What each line means

LINE	WHY IT'S THERE
<code><!DOCTYPE html></code>	Tells the browser "this is modern HTML." Always first. Always.
<code><html lang="en"></code>	Wraps the whole document. <code>lang</code> helps screen readers and search engines.
<code><head></code>	Info <i>about</i> the page — not shown on the page itself.
<code><meta charset="UTF-8"></code>	Lets your page use any character (emoji, accents, Chinese, anything).
<code><meta name="viewport"></code>	Tells phones to render the page at the right size. Without it, mobile sites look tiny.
<code><title></code>	The text in the browser tab and in Google search results.
<code><body></code>	Everything the user actually sees lives here.

HACK — EMMET'S "!" SHORTCUT

In VS Code, type a single `!` on an empty line and press **Tab**. The whole skeleton above appears instantly. This works in any HTML file — try it now.

The save → refresh loop

1. Edit the file in VS Code
2. Save (`Ctrl+S` on Windows, `Cmd+S` on Mac)
3. Switch to the browser and refresh (`F5` or `Ctrl+R`)

That's the entire web-development workflow at this stage. You'll do this thousands of times.

— end of chapter three —

CHAPTER 04

The everyday tags

If you only learn ten tags, learn these. Together they cover 80% of what you'll ever write.

Headings — <h1> through <h6>

```
<h1>The most important heading on the page</h1>
<h2>A section heading</h2>
<h3>A sub-section</h3>
<h4>Smaller still</h4>
```

h1 is the biggest, **h6** the smallest. **Use only one `<h1>` per page** — usually the page title. Don't pick a heading level for its size (CSS will handle that) — pick it for its importance in the outline.

Paragraphs — <p>

```
<p>This is one paragraph of text.</p>
<p>And here is another. Each paragraph appears on its own line.</p>
```

The browser ignores whitespace inside HTML — extra spaces, line breaks, and tabs all collapse to a single space. That's why every paragraph needs its own `<p>` wrapper.

Links — <a>

```
<a href="https://google.com">Go to Google</a>
<a href="about.html">About this site</a>
<a href="mailto:hello@me.com">Email me</a>
<a href="#about">Jump to the About section</a>
```

Four kinds of `href` values you'll use constantly:

- **Full URL** — `https:// ...` — go to another site
- **Relative path** — `about.html` — another file in the same folder
- **Mail link** — `mailto:...` — opens the user's email client
- **Anchor** — `#id` — scroll to an element with that `id`

SECURITY HACK — EXTERNAL LINKS

When you open a link in a new tab with `target="_blank"`, always add `rel="noopener noreferrer"`:

```
<a href="https://google.com" target="_blank" rel="noopener  
noreferrer">Google</a>
```

Without it, the linked page can manipulate yours via JavaScript. Modern browsers default to safe behavior, but adding it is the professional habit.

Images —

```

```

- `src` — path to the image file (relative or full URL)
- `alt` — describes the image for blind users and shows if loading fails. **Never skip this.**
- `width` / `height` — optional, in pixels. Setting them prevents the page from "jumping" as the image loads.

Bold and italic — and

```
<p>This is <strong>very important</strong> and this is <em>emphasized</em>.</p>
```

`<strong>` renders as **bold**. `<em>` renders as *italic*. You'll also see `<b>` and `<i>` — they look the same, but they mean "just style this" instead of "this carries extra meaning." Prefer `strong` / `em` for content; screen readers emphasize them when reading aloud.

Line break —


```
<p>Roses are red<br>Violets are blue</p>
```

Use sparingly — usually a new paragraph is better. `<br>` is right for poetry, mailing addresses, song lyrics. Don't use it to add visual space between paragraphs — that's CSS's job.

Horizontal rule — <hr>

A thematic break in the content — like the line at the end of a scene in a novel.

```
<p>End of part one.</p>  
<hr>  
<p>Beginning of part two.</p>
```

— end of chapter four —

CHAPTER 05

Lists — bullets and numbers

When you have a list of things, use a list. It's a common beginner mistake to use a stack of paragraphs instead.

Unordered list —

Bulleted. Order doesn't matter. Shopping lists, tags, navigation menus.

```
<ul>
  <li>Tomatoes</li>
  <li>Basil</li>
  <li>Olive oil</li>
</ul>
```

Ordered list —

Numbered. Order matters. Cooking steps, instructions, rankings.

```
<ol>
  <li>Boil water</li>
  <li>Add salt</li>
  <li>Drop in the pasta</li>
</ol>
```

You don't type the numbers yourself. The browser numbers them automatically. Reorder the list and the numbers re-number.

Nested lists

A list inside a list. Useful for sub-points.

```
<ul>
  <li>Fruits
    <ul>
      <li>Apple</li>
      <li>Banana</li>
    </ul>
  </li>
  <li>Vegetables</li>
</ul>
```

Description lists — <dl>

A lesser-known list for term / definition pairs. Perfect for glossaries, FAQs, recipe metadata.

```
<dl>
  <dt>Al dente</dt>
  <dd>Pasta cooked so it's still firm to the bite.</dd>

  <dt>Mise en place</dt>
  <dd>Having all your ingredients prepped before you start.</dd>
</dl>
```

WHEN TO USE WHICH LIST

Order matters? . **Just a collection?** . **Pairs of term + meaning?** <dl> . Don't fake a list with paragraphs — search engines and screen readers can tell the difference.

— end of chapter five —

CHAPTER 06

Semantic HTML

"Semantic" means "having meaning." Choose tags that describe what something IS, not just how it should look.

Two pages, same look — different meaning

```
<!-- Bad - every box is a generic div -->  
<div>The top of the page</div>  
<div>Main content</div>  
<div>The footer</div>
```

```
<!-- Good - tags carry meaning -->  
<header>The top of the page</header>  
<main>Main content</main>  
<footer>The footer</footer>
```

Visually identical. But the second version tells **the browser, screen readers, and search engines** what each section is for. That matters because:

- Blind users navigating by screen reader can jump between sections (*"go to main content"*)
- Google ranks pages with clear structure higher
- Your future self reads the code faster

The semantic tags to know

TAG	MEANING
<code><header></code>	Top of a page or section — logo, page title, nav
<code><nav></code>	A group of navigation links
<code><main></code>	The main unique content of the page. One per page.
<code><section></code>	A themed group of content (a chapter, a recipe section)
<code><article></code>	A self-contained piece — a blog post, a recipe, a tweet
<code><aside></code>	Tangentially related content — sidebar, pull quote
<code><footer></code>	Bottom of a page or section — copyright, contact, social links
<code><figure></code> + <code><figcaption></code>	An image (or other content) with a caption
<code><time></code>	A date or time, machine-readable via the <code>datetime</code> attribute

A real page outline

```
<body>
  <header>
    <h1>Tasty Bites</h1>
    <nav>
      <a href="/">Home</a>
      <a href="/recipes">Recipes</a>
    </nav>
  </header>

  <main>
    <article>
      <h2>Carbonara</h2>
      <section><h3>Ingredients</h3> ... </section>
      <section><h3>Steps</h3> ... </section>
    </article>
  </main>

  <footer>
    <p>&copy; 2026 Tasty Bites</p>
  </footer>
</body>
```

If a tag exists for what you're describing, use it. *div* is what you reach for when nothing else fits.

— end of chapter six —

CHAPTER 07

div, span, and friends

Sometimes you need a wrapper that has no meaning — just a box to group things or style. That's what `div` and `span` are for.

<div> — a generic block

A "block" element takes up the full width of its container and starts on a new line. `<div>` wraps related stuff together so CSS or JS can target it.

```
<div class="recipe-card">
  
  <h3>Carbonara</h3>
</div>
```

 — a generic inline piece

An "inline" element only takes up as much space as its content and sits within a line of text. Use `<span>` to highlight a piece of text without changing its flow.

```
<p>Price: <span class="price">$12.99</span></p>
```

Block vs inline — the mental model

BLOCK element (`h1`, `p`, `div`, `section`, ...)
— takes a full line

Another BLOCK element

This is a paragraph with an [INLINE span] inside it flowing along with the regular text.

RULE OF THUMB

Reach for a semantic tag first. If none fits, reach for `<div>` (block) or `<span>` (inline). Never use them when something more meaningful would do.

— end of chapter seven —

CHAPTER 08

Forms — letting users type

Whenever a user types, picks, or clicks something to send data back — that's a form. Logins, searches, signups, contact pages, comment boxes. All the same building blocks.

The minimum form

```
<form>
  <label for="email">Email address</label>
  <input type="email" id="email" name="email">

  <button type="submit">Sign up</button>
</form>
```

Three pieces to know: **input**, **label**, **button**.

Input types — set with type=" ... "

TYPE	WHAT YOU GET
<code>text</code>	Plain single-line text (default)
<code>email</code>	Email-shaped text. Phones show <code>@</code> on the keyboard.
<code>password</code>	Hides characters as dots
<code>number</code>	Numbers only, with up/down arrows
<code>tel</code>	Phone numbers — shows the number keypad on mobile
<code>url</code>	Validates as a URL
<code>date</code>	Native date picker calendar
<code>time</code>	Time picker
<code>color</code>	Native color picker
<code>range</code>	A slider
<code>file</code>	File upload dialog
<code>checkbox</code>	A tickbox
<code>radio</code>	Pick one of many (group with the same <code>name</code>)
<code>search</code>	Search box (with a clear "x" in some browsers)
<code>submit</code>	A submit button

Why labels matter

The `<label>` tag connects descriptive text to an input. Connect them by matching the label's `for` attribute to the input's `id` .

```
<label for="username">Username</label>
<input type="text" id="username" name="username">
```

Why bother?

- **Clicking the label focuses the input** — bigger target, less friction
- **Screen readers read the label** when reading the input
- **SEO and accessibility audits** reward you

Useful input attributes

ATTRIBUTE	WHAT IT DOES
<code>placeholder</code>	Grey hint text shown until the user types
<code>required</code>	Form won't submit unless this is filled
<code>value</code>	Default value
<code>maxlength</code> / <code>minlength</code>	Character limits
<code>min</code> / <code>max</code> / <code>step</code>	For number, date, range
<code>pattern</code>	A regex the value must match
<code>autocomplete</code>	Tell browsers what to autofill: <code>email</code> , <code>name</code> , <code>tel</code> , <code>current-password</code>
<code>autofocus</code>	Focus this field automatically on page load
<code>disabled</code>	Greyed out, can't be edited or submitted
<code>readonly</code>	Visible value, not editable, still submitted

Other form controls

Multi-line text — `<textarea>`

```
<label for="msg">Your message</label>
<textarea id="msg" name="message" rows="5"></textarea>
```

Dropdown — `<select>`

```
<label for="meal">Favorite meal</label>
<select id="meal" name="meal">
  <option value="breakfast">Breakfast</option>
  <option value="lunch" selected>Lunch</option>
  <option value="dinner">Dinner</option>
</select>
```

Checkboxes and radios

```
<!-- Checkboxes - pick zero or many -->  
<label><input type="checkbox" name="diet" value="vegan"> Vegan</label>  
<label><input type="checkbox" name="diet" value="gluten-free"> Gluten-  
free</label>  
  
<!-- Radios - pick exactly one. Same name = same group. -->  
<label><input type="radio" name="size" value="small"> Small</label>  
<label><input type="radio" name="size" value="large"> Large</label>
```

TRICK — WRAP INPUTS IN LABELS

For checkboxes and radios, putting the `<input>` *inside* the `<label>` means you can skip `for` / `id` entirely. Clicking the label still focuses the input.

Group related fields — `<fieldset>`

```
<fieldset>  
  <legend>Size</legend>  
  <label><input type="radio" name="size"> Small</label>  
  <label><input type="radio" name="size"> Medium</label>  
  <label><input type="radio" name="size"> Large</label>  
</fieldset>
```

`<fieldset>` draws a box around related fields. `<legend>` labels the group.

Buttons

```
<button type="submit">Send</button>  
<button type="reset">Clear</button>  
<button type="button">Do something</button>
```

Three button types. Inside a form, the default is `submit` — so always set `type="button"` on any button that *shouldn't* submit the form (otherwise Enter in any input will trigger it).

— end of chapter eight —

CHAPTER 09

Tables — for actual tabular data

Tables are for data in rows and columns. Not for layout. Layout uses CSS Grid and Flexbox.

The shape of a table

```
<table>
  <thead>
    <tr>
      <th>Recipe</th>
      <th>Time</th>
      <th>Category</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Carbonara</td>
      <td>25 min</td>
      <td>Dinner</td>
    </tr>
  </tbody>
</table>
```

TAG	WHAT IT IS
<table>	The outer wrapper
<thead>	The header row group
<tbody>	The data row group
<tr>	Table row
<th>	Header cell (bold + centered by default)
<td>	Data cell
<caption>	Optional title for the table

DON'T USE TABLES FOR LAYOUT

Years ago, designers built whole webpage layouts using nested tables. Don't. Tables tell screen readers *"this is tabular data"* — which confuses everything when it's actually a sidebar. Use CSS for layout, tables for data.

— end of chapter nine —

CHAPTER 10

Multi-page sites

Real websites have more than one page. You make them by creating more HTML files and linking them.

One folder, several files

```
my-site/
├── index.html      ← the homepage (always named "index.html")
├── about.html
├── contact.html
├── images/
│   ├── logo.png
│   └── hero.jpg
```

Use `<a>` to link between them:

```
<a href="about.html">About us</a>
<a href="images/logo.png">View our logo</a>
```

Why index.html is special

When someone visits a folder URL (like `mysite.com/` or `mysite.com/recipes/`), the server looks for a file called `index.html` and serves it. That's why your homepage is always named `index.html`.

Paths — relative vs absolute

PATH	WHAT IT MEANS
<code>about.html</code>	Same folder as the current file
<code>recipes/carbonara.html</code>	Subfolder named <code>recipes</code>
<code>../about.html</code>	One folder UP, then about.html. <code>..</code> means "parent folder."
<code>/about.html</code>	"Root" of the site — starts at the top no matter where you are
<code>https://example.com/</code>	An absolute URL — a full address to anywhere on the web

— end of chapter ten —

CHAPTER 11

Tips & little hacks

Small habits that separate clean HTML from messy HTML. Steal these and your code will feel professional from day one.

Use Emmet shortcuts in VS Code

VS Code has built-in Emmet — type a tag name and press **Tab**:

- `p` → `<p></p>`
- `ul>li*3` → an unordered list with 3 list items
- `nav>a*4` → a nav with 4 anchor tags
- `.recipe-card` → `<div class="recipe-card"></div>`
- `#header` → `<div id="header"></div>`
- `!` → the full HTML5 skeleton
- `lorem20` → 20 words of placeholder Latin

Always set the language

The `lang` attribute on `<html>` tells screen readers what language to pronounce — and tells search engines what language the page is.

```
<html lang="en">
<html lang="en-US">
<html lang="es">
```

Always include the viewport meta tag

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Without it, mobile browsers render your page at desktop width and zoom out — making everything tiny.

Always write alt text

For meaningful images: describe what's in it.

For decorative images (icons, dividers): use empty `alt=""` so screen readers skip them.

```
<!-- Meaningful -->


<!-- Decorative -->

```

HTML entities — write special characters

Some characters can't be typed directly because the browser would think they're HTML. Use entities — they start with `&` and end with `;`:

ENTITY	RENDERS AS	USE FOR
<code>&amp;</code>	<code>&</code>	The literal ampersand
<code>&lt;</code> / <code>&gt;</code>	<code><</code> <code>></code>	Less than / greater than
<code>&copy;</code>	©	Copyright
<code>&trade;</code>	™	Trademark
<code>&reg;</code>	®	Registered
<code>&nbsp;</code>		A non-breaking space (text won't wrap here)
<code>&mdash;</code>	—	An em dash
<code>&ndash;</code>	–	An en dash
<code>&hellip;</code>	...	Ellipsis

Tip: emoji work natively now — you can just paste them. `<p>Hello 🌈</p>` works fine.

Make phone numbers and emails tappable

```
<a href="tel:+1234567890">Call us</a>
<a href="mailto:hello@me.com?subject=Hi">Email</a>
<a href="sms:+1234567890">Text us</a>
```

On phones, these open the dialer, mail app, and messages app respectively.

Force a download instead of opening

```
<a href="recipe.pdf" download>Download recipe</a>
<a href="recipe.pdf" download="carbonara-recipe.pdf">Download</a>
```

The `download` attribute makes the browser save instead of open. Add a value to rename the file on save.

Data attributes — stash data on any tag

Any attribute starting with `data-` is your own custom data. JavaScript can read it back. CSS can use it for styling.

```
<button data-category="dinner" data-recipe-id="42">
  Carbonara
</button>
```

Native expand/collapse — <details>

```
<details>
  <summary>Click to see the recipe</summary>
  <p>Boil pasta. Mix eggs and cheese. Combine.</p>
</details>
```

A toggleable section that works without any JavaScript. Great for FAQs, optional details, mobile menus.

The <dialog> element — native modals

```
<dialog id="my-modal">
  <p>Are you sure?</p>
  <button onclick="this.closest('dialog').close()">Cancel</button>
</dialog>
```

A real dialog box, focus-trapped, ESC closes it, all without JS libraries.

Mark important text — <mark>

```
<p>Don't skip the <mark>resting time</mark> — it matters.</p>
```

Renders with a highlighter-like background. Good for search results too.

Show keyboard shortcuts — <kbd>

```
<p>Press <kbd>Ctrl</kbd> + <kbd>S</kbd> to save.</p>
```

Group form fields visually

Wrap label + input pairs in a `<div>` so CSS can space them out:

```
<div class="field">
  <label for="email">Email</label>
  <input type="email" id="email">
</div>
```

autocomplete saves users typing

```
<input type="text"      autocomplete="name">
<input type="email"    autocomplete="email">
<input type="tel"      autocomplete="tel">
<input type="password" autocomplete="current-password">
```

The browser pre-fills these from saved info. Users love this — they won't even know your form is well-written, just that filling it is fast.

Use semantic time

```
Published <time datetime="2026-06-15">June 15, 2026</time>
```

The `datetime` attribute is machine-readable. Screen readers, search engines, and calendars all understand it.

The `<picture>` element — different images for different screens

```
<picture>
  <source media="(max-width: 600px)" srcset="small.jpg">
  <source media="(max-width: 1200px)" srcset="medium.jpg">
  
</picture>
```

Phone users download `small.jpg`, desktop users download `large.jpg`. Saves bandwidth.

Loading lazy on images below the fold

```

```

The browser only loads the image when the user scrolls near it. Free performance win.

— end of chapter eleven —

CHAPTER 12

Common mistakes

Every beginner makes these. Spotting them early saves hours of confusion.

1. Forgetting to close tags

```
<!-- Wrong -->
<p>This paragraph never ends
<p>So this one joins it weirdly</p>
```

2. Using headings for size, not meaning

Don't write `<h3>` just because you want smaller text. Pick the heading level based on outline depth. Use CSS for size.

3. Multiple `<h1>`s on one page

One per page. The browser allows more, but it hurts SEO and accessibility.

4. Skipping alt text

The most common accessibility failure. `alt=""` for decoration is fine. Just don't leave it off entirely.

5. `<br>` as spacing

```
<!-- Bad -->
<p>Paragraph one.</p>
<br><br>
<p>Paragraph two.</p>
```

Use CSS `margin` for space between elements. `<br>` is for line breaks within text only.

6. div for everything

Code that's all `<div>` is a sign you haven't picked the right semantic tags. Use `header`, `nav`, `main`, `article`, `section`, `footer` when they fit.

7. Inline styles for everything

```
←!— Bad —→  
<p style="color: red; font-size: 18px;">Hello</p>
```

Inline styles are hard to change later. Put styling in a separate CSS file. Use `class` on the HTML.

8. Forgetting the viewport meta tag

Without it, your site looks tiny on phones. Always include it in `<head>`.

9. Buttons that aren't `<button>`

If it's clickable and does something, make it a real `<button>`. Don't use `<div onclick="...">`. Buttons have keyboard support, focus styles, and screen-reader announcements built in.

10. Reusing the same `id`

`id` values must be unique per page. If you need to reuse, use `class` instead.

— end of chapter twelve —

CHAPTER 13

The complete tag cheat sheet

Bookmark this page. Print it. Tape it to the wall. Every tag you've met in the course, in one place.

Document structure

`<html>` root

`<body>` visible content

`<meta>` page metadata

`<script>` load JS

`<head>` page info

`<title>` tab text

`<link>` load CSS / fonts

`<style>` inline CSS

Semantic regions

`<header>` top of page

`<main>` main content

`<article>` self-contained

`<footer>` bottom of page

`<nav>` navigation

`<section>` thematic group

`<aside>` sidebar

`<figure>` media + caption

Text content

`<h1>`–`<h6>` headings

`<strong>` important

`<b>` stylistic bold

`<u>` underline

`<small>` fine print

`<code>` inline code

`<kbd>` keyboard input

`<cite>` source title

`<q>` inline quote

`<br>` line break

`<p>` paragraph

`<em>` emphasis

`<i>` stylistic italic

`<mark>` highlight

`<sub>` / `<sup>` sub/super

`<pre>` preformatted

`<abbr>` abbreviation

`<blockquote>` quote block

`<time>` date/time

`<hr>` horizontal rule

Lists

`<ul>` unordered list

`<ol>` ordered list

`<li>` list item

`<dl>` description list

`<dt>` description term

`<dd>` description detail

Media

`<img>` image

`<picture>` responsive image

`<source>` media source

`<audio>` sound

`<video>` video

`<track>` captions

`<iframe>` embed

`<figcaption>` caption

Links & navigation

`<a href>` link

`<a href="#id">` in-page jump

`<a href="mailto:">` email

`<a href="tel:">` phone

Forms

`<form>` form wrapper

`<input>` single field

`<label>` field label

`<textarea>` multi-line

`<select>` dropdown

`<option>` dropdown option

`<optgroup>` grouped options

`<button>` button

`<fieldset>` group fields

`<legend>` fieldset title

`<datalist>` autocomplete list

`<output>` result of calculation

Tables

`<table>` the table

`<caption>` table title

`<thead>` header rows

`<tbody>` body rows

`<tfoot>` footer rows

`<tr>` row

`<th>` header cell

`<td>` data cell

Generic / interactive

`<div>` block container

`<span>` inline container

`<details>` expand/collapse

`<summary>` expander title

`<dialog>` native modal

`<template>` reusable HTML

— end of chapter thirteen —

CHAPTER 14

Where to go next

You now know enough HTML to build any web page's structure. The journey from here.

The natural next steps

1. **CSS** — color, fonts, spacing, layout. The "looks like a real site" part. (*Course Lesson 03.*)
2. **Flexbox & Grid** — modern CSS layout. Cards in rows, headers with logos and nav side by side. (*Lesson 04.*)
3. **JavaScript** — interactivity. Buttons that do things, forms that validate, content that updates. (*Lessons 05–06.*)
4. **TypeScript** — JavaScript with safety rails. (*Lesson 07.*)
5. **Tailwind CSS** — a faster way to style. (*Lesson 08.*)

Resources worth bookmarking

- **MDN Web Docs** (developer.mozilla.org) — the official, most accurate reference for every HTML tag.
- **HTML5 Doctor** — short articles on semantic HTML choices.
- **Can I Use** (caniuse.com) — check which browsers support a tag or attribute.
- **The W3C HTML Validator** — paste your HTML, get a list of errors.

Build something — anything

Reading about HTML teaches you the vocabulary. Building teaches you to write. Don't wait until you "know enough" — you'll never feel ready. Pick something tiny:

- A one-page bio about yourself
- A page that lists your three favorite movies, each with an image
- A simple recipe page like the one we built in class
- A contact form for an imaginary business

The fastest way to learn HTML is to build five small, ugly websites in a week. The second-fastest is to build one. Don't read another tutorial first — open VS Code now.

— end —

• • •

Tasty Bites · Frontend Cohort